

参考：

- 《Java 虚拟机垃圾回收机制》 [链接](#)

垃圾回收 - 概述

GC 的内容

- GC 回收的是 **堆内存** 中的对象
- 当一个对象没有任何引用的时候，就会被垃圾回收器所回收（循环依赖不算引用）

GC 的时机

- GC 一般是由 JVM 自动执行，我们没有办法去手动调用 GC
- GC 只有在堆内存不够的时候，才会触发
- 如果我们希望回收一个对象的时候，可以将该对象置空、或者使用软引用、虚引用的方式，**加速对象的回收**
- **`System.gc()`** 和 **`Runtime.gc()`** 会向 JVM 发送执行 GC 请求，但是 JVM 不保证一定会执行 GC

GC 的影响

- 当 Activity 被回收的时候，会触发它的 **`onTrimMemory()`** 方法
- 当一个对象被回收的时候，会触发它的 **`finalize()`** 方法

GC 的线程

- GC 是由一个被称为 **垃圾回收器的守护线程** 执行的
- 一旦触发 GC, **所有线程都会停止** (频繁的 GC, 会造成卡顿)

垃圾回收 - 哪些对象需要回收

堆中哪些对象需要回收, 一般有 2 种策略 (**垃圾标记算法**)

- 引用计数器法
- 可达性分析: JVM 虚拟机采用

引用计数法

引用计数法 (Reference Counting) : 给对象添加一个 **引用计数器**, 每当有一个地方引用它, 计数器加1, 引用失效, 计数器减1; 当计数器的值为0时, 即可回收该对象

- 优点: 实现简单, 效率高
- 缺点: 无法解决对象 **互相引用** 的问题, **无法区分软、虚、弱、强引用类别**

```
public void main() {  
    A a = new A();           // A的引用+1 = 1  
    B b = new B();           // B的引用+1 = 1  
  
    a.instance = b;          // B的引用+1 = 2  
    b.instance = a;          // A的引用+1 = 2  
  
    a = null;                 // A的引用-1 = 1  
    b = null;                 // B的引用-1 = 1  
}
```

可达性分析

可达性分析 (Reachability Analysis) :

- 定义一系列的 **GC Roots** 对象作为根节点
- 从这些根节点开始向下搜索，搜索所走过的路径称为 **引用链** (Reference Chain)
- 如果没有任意一条引用链，可以让一个对象到达 GC Roots 的话，即可回收该对象

GC Roots 对象包括

- **虚拟机栈** 栈帧中局部变量表引用的对象
- **本地方法栈** 中 JNI 引用的对象
- **方法区的常量池中** 引用的对象
- **方法区的静态区** 引用的对象

GC 自救 - finalize 方法

当一个对象被回收的时候，它会触发 `finalize` 方法，并且 `finalize` 方法只会执行一次

- `finalize` 方法在第一次自救的时候就已经执行了，第二次是直接回收而不会再调用 `finalize` 方法

在可达性分析中不可达的对象，并不是立即死亡，而是 **经过 2 次标记后再死亡**（每次标记后都会进行一轮回收）

第一次标记：**是否有必要执行 `finalize` 方法**

- 可达性分析中不可达的对象，在第一次标记的时候会进行一次筛选，筛选的条件是"是否有必要执行 `finalize` 方法"
- **没有覆盖 `finalize` 方法、`finalize` 方法已经被执行过一次**，这 2 种情况被视为没有必要执行

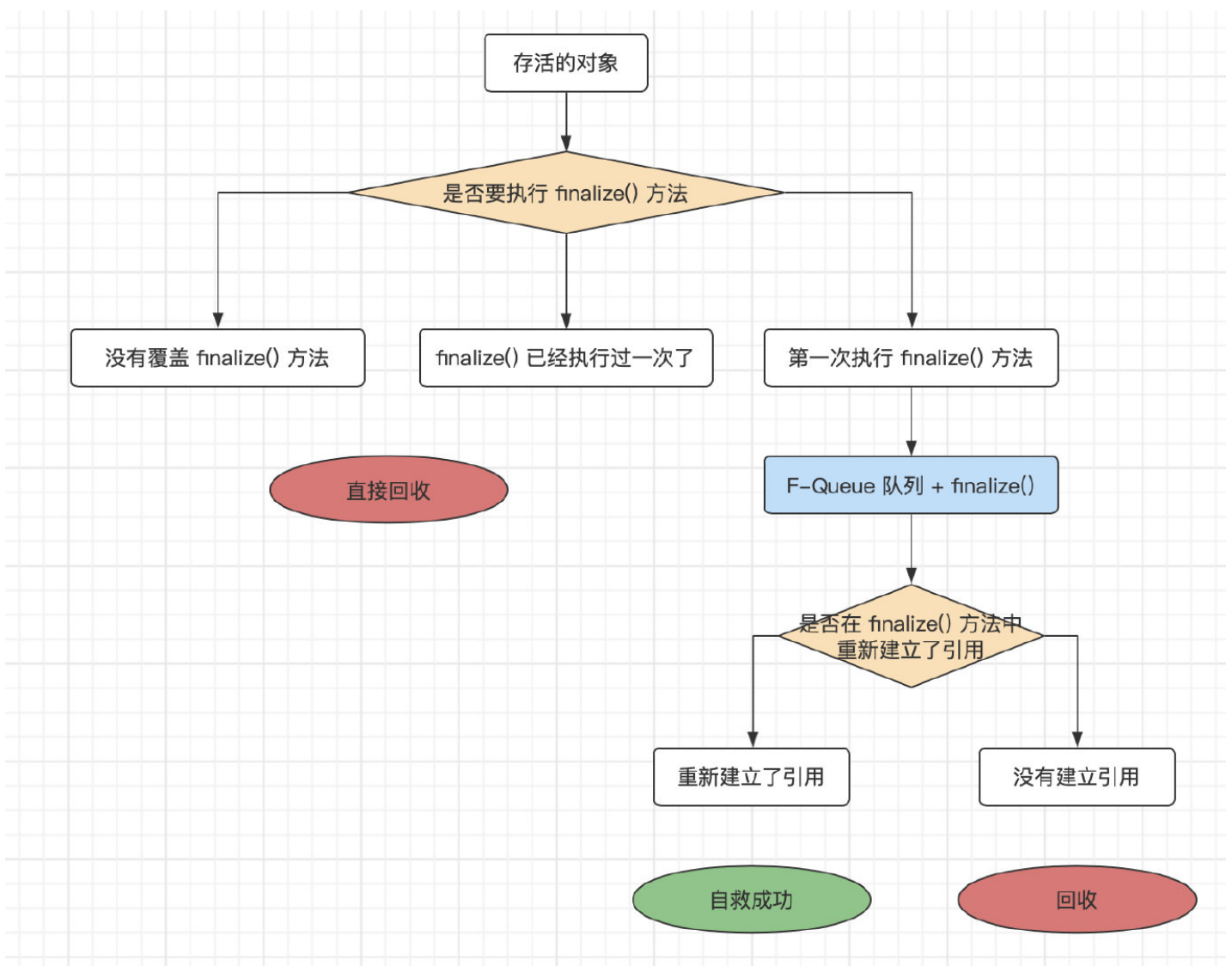
第一次标记结果：**F-Queue 队列**

- 第一次标记筛选后，将需要执行 `finalize` 方法的对象放入一个 F-Queue 的队列中；不需要执行的，直接进行回收，不会再触发 `finalize` 方法
- 稍后 JVM 会开启一个低优先级的 **Finalizer 线程** 去执行 F-Queue 队列中的对象（虚拟机并不会等待该线程执行完毕，

因为有可能一个对象的 `finalize` 方法执行缓慢、或者死循环，导致下面的代码不会执行)

第二次标记： **是否在 `finalize` 方法中，重新建立了引用**

- 第二次标记是 GC 对 F-Queue 队列中的对象进行一次标记筛选，筛选的条件是"是否在 `finalize` 方法中重新建立了引用"
- 如果对象在 `finalize` 方法中，重新建立引用链，那么就会将该对象移出可回收队列
- 也叫 **GC 自救**，这种 **自救的机会只有一次**，因为 `finalize` 方法只会执行一次（下次 GC 会直接回收）



GC 自救

```
public class FinalizeGC {

    public static FinalizeGC mInstance = null;

    @Override
    protected void finalize() throws Throwable {
        super.finalize();
        System.out.println("finalize");
        FinalizeGC.mInstance = this;        // GC 自救
    }

    public static void main(String[] args) throws Throwable {
        FinalizeGC.mInstance = new FinalizeGC();

        // 第一次成功解救了自己
        FinalizeGC.mInstance = null;
        System.gc();                        // 执行 finalize() 方法，在 Finalizer 线程中执行

        Thread.sleep(500);                  // 因为 Finalizer 线程是低优先级的（异步），所以这里等待了 500ms 等待自救成功
        if(FinalizeGC.mInstance == null) {
            System.out.println("死了");
        } else {
            System.out.println("活着");
        }

        // 第二次自救失败
        FinalizeGC.mInstance = null;
        System.gc();                        // 不会执行 finalize() 方法
        Thread.sleep(500);
    }
}
```

```
if(FinalizeGC.mInstance == null) {  
    System.out.println("死了");  
} else {  
    System.out.println("活着");  
}  
}  
}
```

// 输出 finalize-活着-死了

垃圾回收器 - 如何回收

如何回收：也就是垃圾回收机制、**垃圾回收算法**

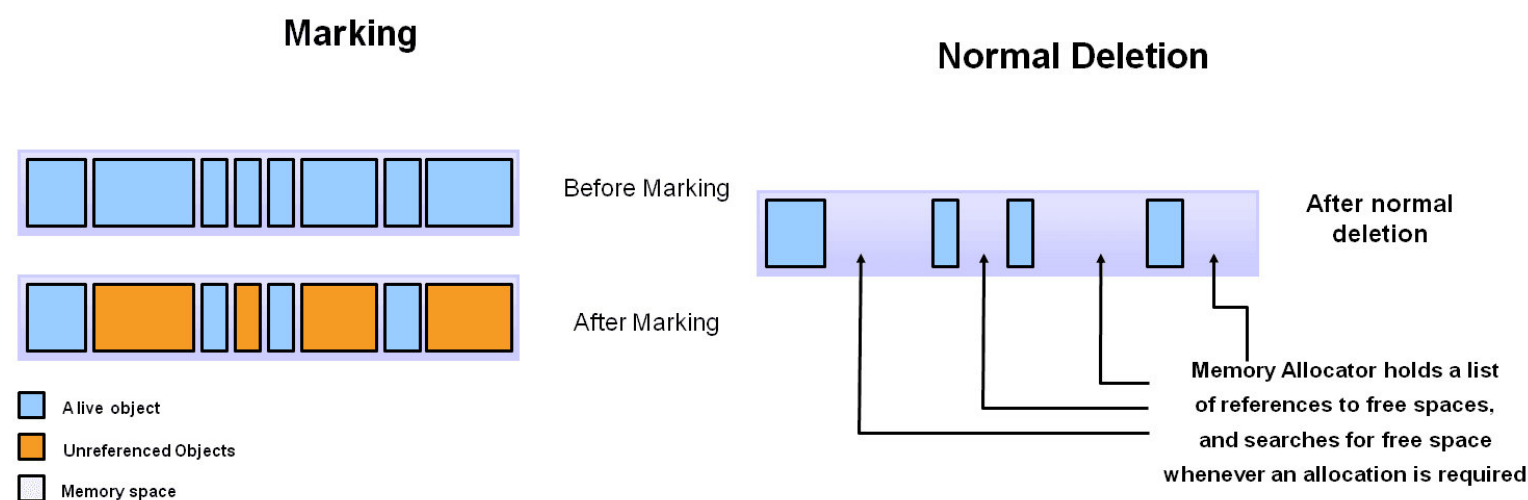
算法的选择：**一般新生代采用复制算法，老年代采用标记整理算法或标记清除算法**

- 复制算法，适合 **对象存活率低** 的内存空间
 - 在对象存活率高的时候，复制算法会进行大量的复制操作，效率低；
 - 并且会造成内存空间的极大浪费，如果不想浪费那 50% 的空间，需要额外的空间进行一个分配担保
- 标记整理算法、标记清除算法，适合 **对象存活率高** 的内存空间

标记清除算法

标记清除算法（Mark-Sweep）：首先标记出所有需要回收的对象，标记完成后进行统一的回收

- 缺点：标记和清除 2 个步骤，**效率都不高**（所有堆内存中堆对象都会被扫描一遍）
- 缺点：会存在大量不连续的**内存碎片**，当需要分配大内存对象时，不得不提前进行一次垃圾回收



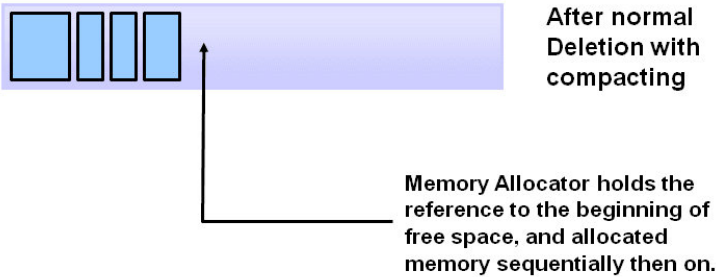
标记整理算法

标记整理算法（Mark-Compact）：首先标记出所有需要回收的对象，标记完成后将所有存活对象**统一向一端移动**（压缩内存空间），然后清除掉边界以外的内存

- 优点：解决了内存碎片的问题

- 缺点：依然存在 **标记效率不高** 的问题

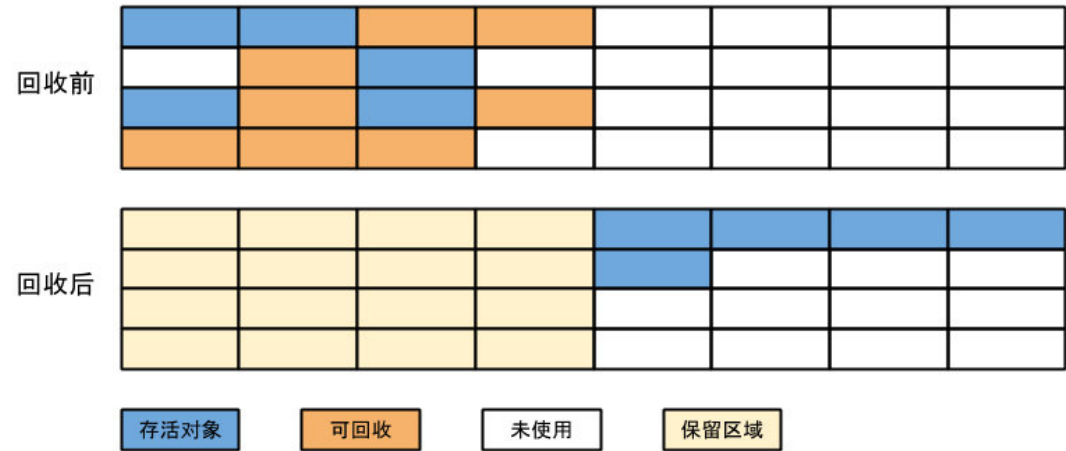
Deletion with Compacting



复制算法

复制算法（Copy）：将内存分为 **大小相等的 2 块空间**，每次只使用其中一块，GC 的时候，将存活的对象从一块内存空间复制到另一块，然后把已使用的空间清除掉

- 优点：解决了标记清除算法中效率不高，以及内存碎片的问题
- 缺点：造成了 **内存空间的极大浪费**



分代收集算法

分代收集算法（Generation Collection）：根据对象 **存活周期** 的不同，将内存空间划分为 **新生代**、**老年代**、**永久代**

- 根据每个年代各自的特点，分别采用不同的收集算法
- 当代商业虚拟机都采用分代收集算法

新生代 Young Generation：对象存活率低（只需要复制少量对象，复制成本低），采用 **复制算法**

- 我们会将新生代又分为 **Eden**、**Survivor1**、**Survivor2** 三个区域，以提高内存利用率

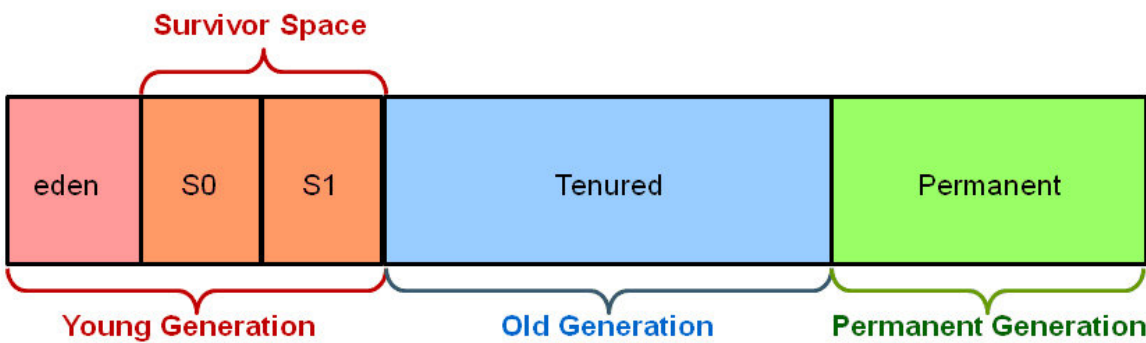
老年代 Tenured/Old Generation：对象存活率高（没有额外的空间进行分配担保），采用 **标记清除算法**、或者 **标记整理算法**

- 一些 **大的字符串、数组**，可能会直接存储到老年代中

永久代 Permanent Generation：对象存活率极高（比如 **方法区** 中的数据）

- 方法区也会进行垃圾回收，只是频率低（效率低）
- 方法区对应着永久代，一般需要回收 **常量** 和 **无用的类信息**

Hotspot Heap Structure



分代收集算法

内存分代

Heap Space						Method Area		Native Area					
Young Generation				Old Generation		Permanent Generation		Code Cache					
Virtual	From Survivor 0	To Survivor 1	Eden	Tenured	Virtual	Runtime Constant Pool		Thread 1..N			Compile	Native	Virtual
						Field & Method Data		PC	Stack	Native Stack			
						Code							

新生代

新生代用来存储那些 **新 new 出来的对象**

Minor Collections：发生在新生代的垃圾收集称为 Minor collections（小收集）

- 当新生代这部分内存满了之后，就会发起一次垃圾收集事件
- 这种收集通常 **比较快**，因为新生代的大部分对象都是需要回收的，那些暂时无法回收的就会被移动到老年代
- 所有小收集都是 **全局暂停事件**（Stop the World），这意味着 **所有的应用线程都需要停止**，直到垃圾回收的操作全部完成

新生代的优化（复制算法的优化）

- 划分 Eden 和 Survivor 空间
 - 新生代中 98% 的对象都是朝生夕死的，因此不需要按照 1:1 的关系划分内存空间
 - 我们将新生代划分为 **1 块较大的 Eden 空间**（存储新 new 出来的对象）和 **2 块较小的 Survivor 空间**（存储存活的对象），一般是 8:1:1 的关系
 - 每次使用 Eden 和其中一块 Survivor（空间利用率 90%），在 GC 时，将 Eden 和 Survivor1 中存活的对象复制至另一块 Survivor 空间
- 当新生代空间不够时，需要依赖老年代进行一个 **分配担保**

老年代

老年代用来存储那些 **存活时间较长** 的对象（一些 **大的字符串、数组**，可能会直接存储到老年代中）

- 一般来说，我们会给新生代的对象限定一个存活的时间，当达到这个时间还没有被收集的时候就会被移动到老年代中

Major Collections：发生在老年代的垃圾收集称为 Major Collections（大收集）

- 随着时间的推移，老年代也会被填满，最终导致老年代也要进行垃圾回收
- 通常大收集 **比较慢**，通常是 Minor Collection 的 10 倍，因为它涉及到所有的存活对象（对于时间要求高的应用，应该将大收集最小化）
- 大收集也是 **全局暂停事件**（暂停时长会受到用于老年代的垃圾回收器的影响）

老年代的优化

- **大对象直接进入老年代**：为了避免大对象在 Eden 区和 Survivor 区发生大量的复制操作，规定大对象直接进入老年代
 - 大对象：需要占用大量连续内存空间，比如 **数组、大字符串** 等等
- **动态年龄判定**：如果 Survivor 空间中，某个年龄对象的总和大于 Survivor 空间的一半，那么大于或等于该年龄的所有对象，直接进入老年代（无视年龄阈值的限制）
- **年龄阈值**：长期存活的对象进入到老年代

永久代

永久代用来存储 **类和方法的元数据**（常量、静态变量等等）

- 当永久代发现有些类不再被其他类所需要，并且其他类需要更多空间当时，这些类的元数据就有可能被回收

上面讨论的都是堆中对象的回收，很多人都认为 **方法区**（永久代）是没有垃圾收集的，但其实方法区中也存在着垃圾回收

- 性价比低，堆中的新生代回收一次可回收 70-85% 的内存

方法区中，主要回收废弃的常量和无用的类

- **废弃常量**：同对象的回收类似（没有任何引用，即回收）
- **无用的类**：该类的 **所有实例** 已经被回收、该类的 **Class 对象** 不存在任何使用、该类的 **类加载器** 已经被回收，3 者缺一不可
- **静态区** 是不被回收的

空间分配担保

空间分配担保：新生代采用复制算法的时候，由于内存原因有一定几率造成 GC 失败，所以需要额外的空间进行担保

- 在发生 Minor GC 之前，虚拟机会先检查 **老年代的最大可用连续空间** 是否大于 **新生代对象的总和**

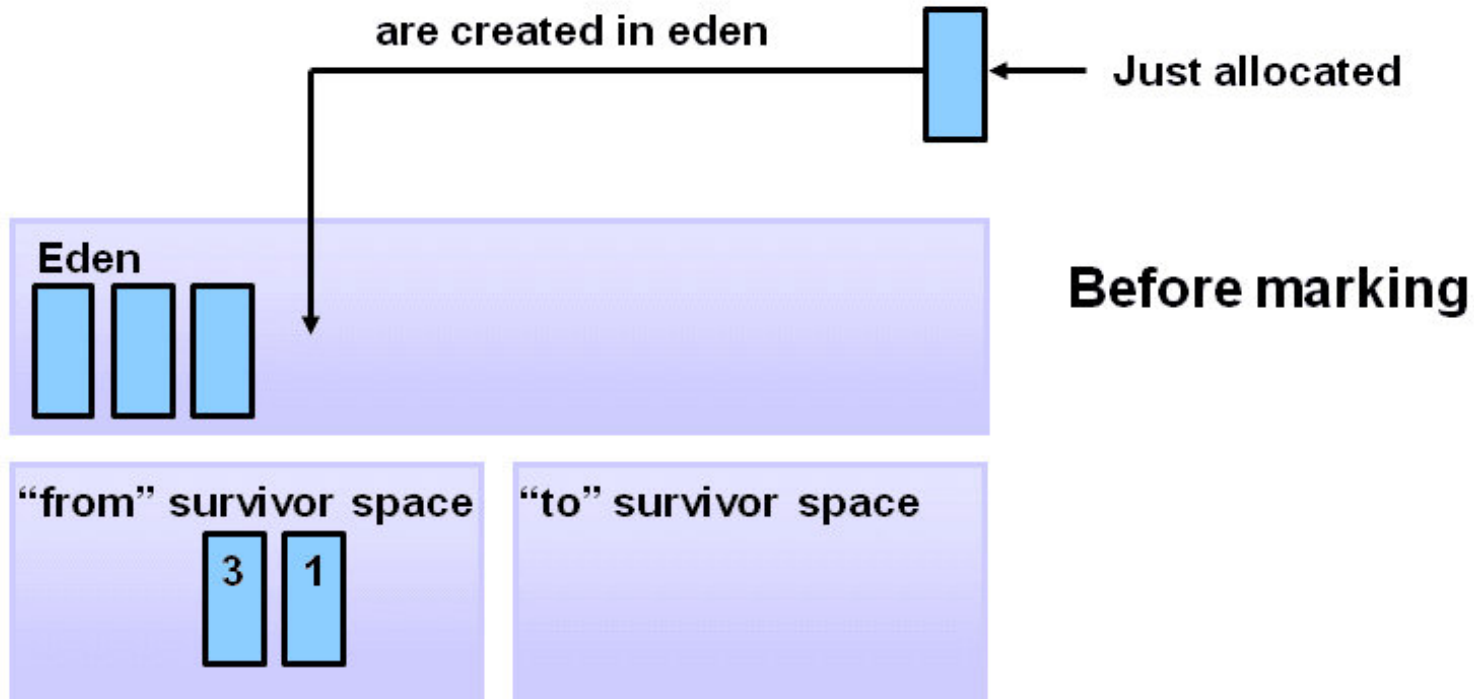
- 如果大于，那么直接进行一次 Minor GC
- 如果小于，那么会检查 HandlePromotionFailure 设置，是否允许担保失败
- 如果允许担保失败，那么会继续检查 老年代的最大可用连续空间 是否大于 历次晋升到老年代对象的平均大小
 - 如果大于，那么会尝试进行一次 Minor GC（尽管这次是有风险的）
 - 如果小于，或者 HandlePromotionFailure 设置不允许冒险，那这时会改为进行一次 Major GC

在 JDK 6 之后，HandlePromotionFailure 开关的打开与否，不影响分配担保机制

垃圾回收过程

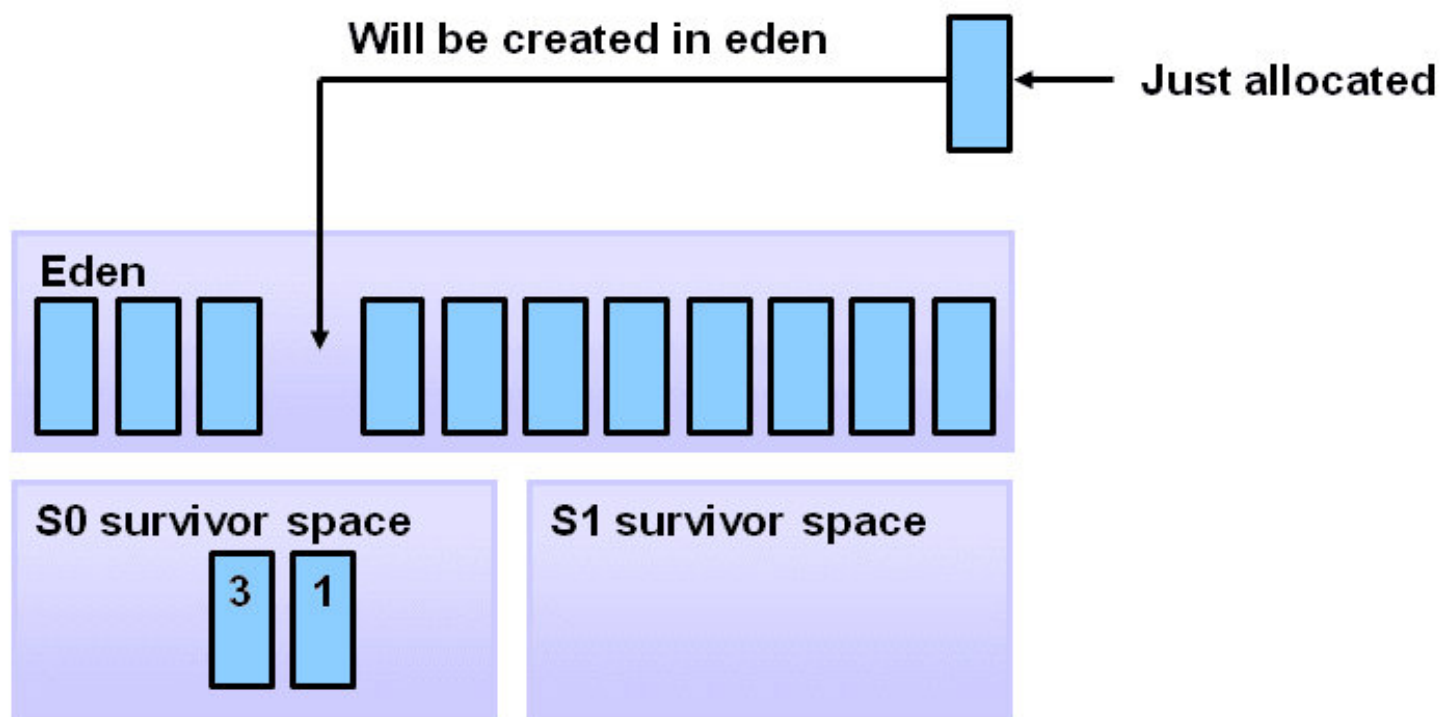
1. 所有 new 出来的对象都会优先分配到新生代的 Eden 区，2 个 Survivor 区域初始化的时候是空的

Object Allocation



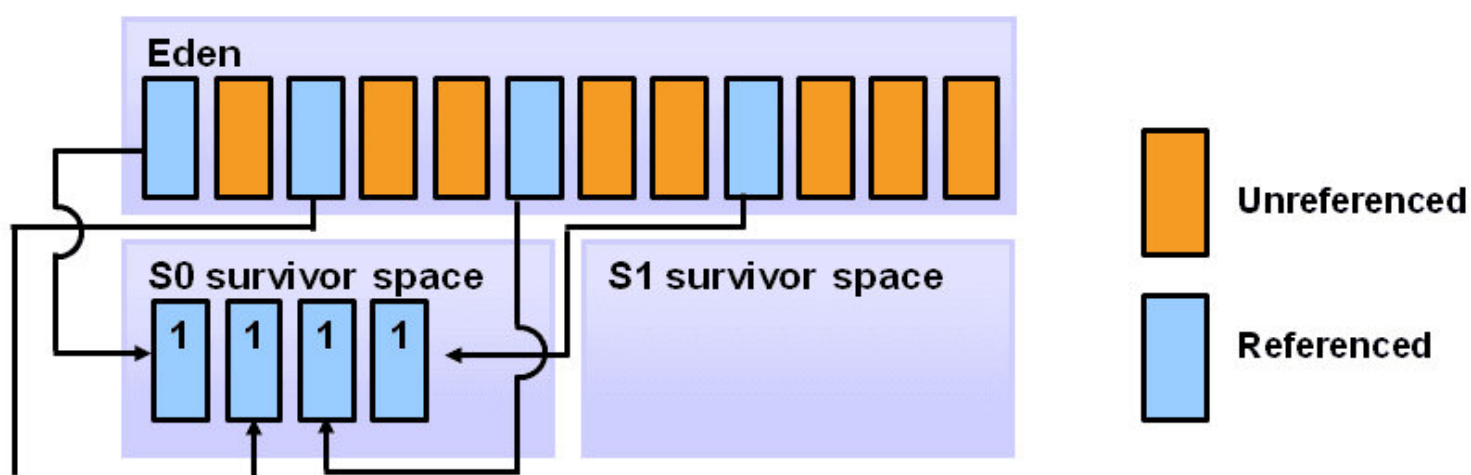
2. 当 Eden 区满了之后，就会触发一次 **小收集 Minor Collection**

Filling the Eden Space



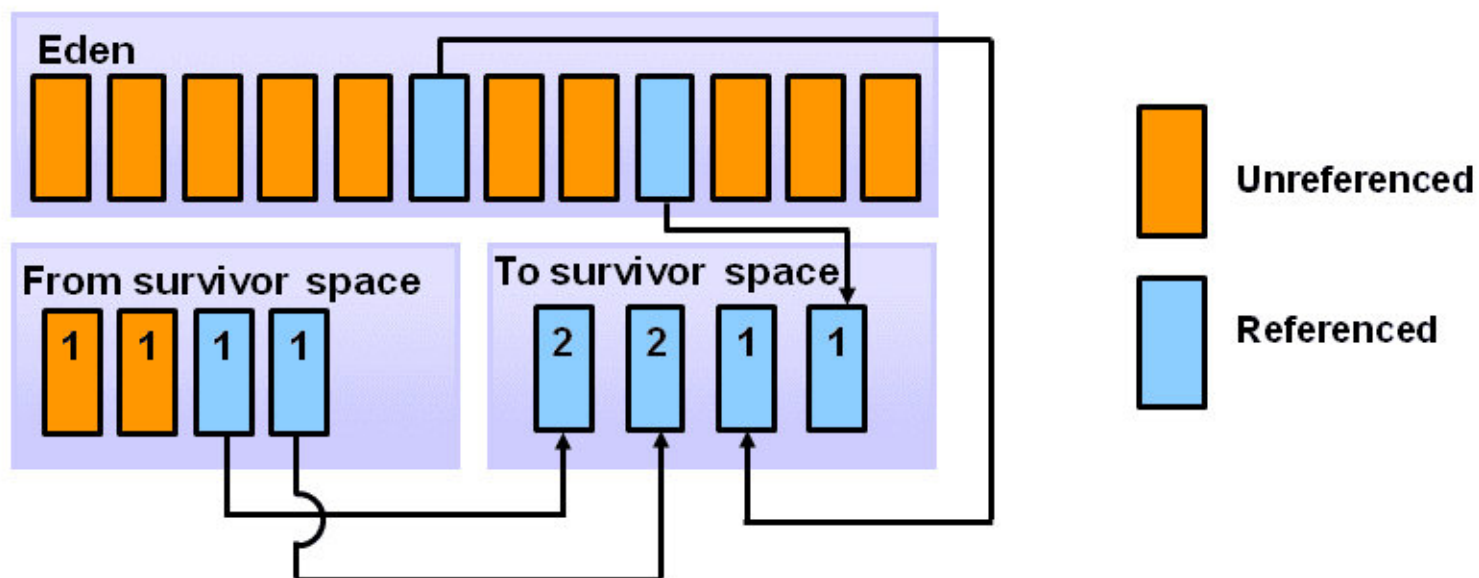
3. 小收集存活下来的对象会被移到 **Survivor0 区域**

Copying Referenced Objects



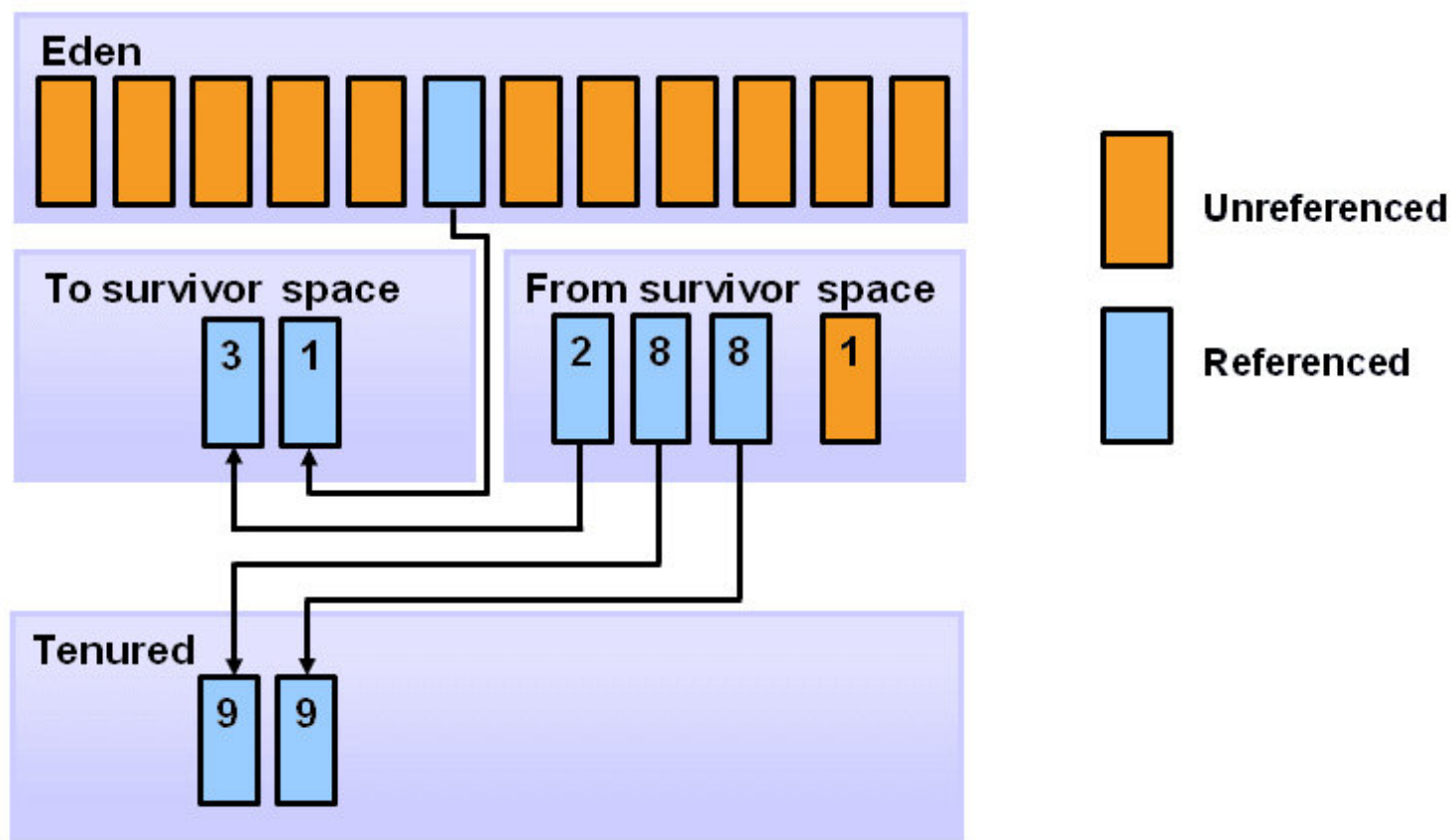
4. 当 Eden 区又填满的时候，又会发生下一次垃圾回收，这次存活的对象会被移动到 **Survivor1 区域**（包括 Eden 区和 Survivor0 区）

Object Aging



5. 之后会一直重复以上步骤，并记录对象的年龄，当年龄达到一定 **阈值** 时，就将新生代中的对象移动到 **老年代** 中

Promotion



6. 之后会一直重复以上步骤，不断的进行 **对象的清除和年代的移动**
7. 大部分的垃圾回收都发生在新生代，只有当老年代中的内存不够了，才会发生一次 **大收集 Major Collection**
- 老年代的内存空间不够了
 - 老年代的空间分配担保失败
-

垃圾回收 - 垃圾回收器

JVM 中的垃圾收集一般采用 **分代收集算法**，不同的堆内存区域采用不同的收集算法（Java 提供了多种垃圾回收器），主要目的是 **增加吞吐量**（运行用户代码的时间/总时间）、**降低停顿时间**

1. **Serial 收集器**： **新生代收集器**，使用复制算法
 - 使用一个线程进行 GC，串行，其它工作线程暂停。
2. **ParNew 收集器**： **新生代收集器**，使用复制算法
 - Serial 收集器的多线程版，用多个线程进行 GC，并行，其它工作线程暂停
 - 使用 `-XX:+UseParNewGC` 开关来控制使用
 - ParNew + Serial Old 收集器组合收集内存；使用 `-XX:ParallelGCThreads` 来设置执行内存回收的线程数
3. **Parallel Scavenge 收集器**： **新生代收集器**，使用复制算法
 - 吞吐量优先的垃圾回收器，关注 CPU 吞吐量
 - 使用 `-XX:+UseParallelGC` 开关控制使用
 - Parallel Scavenge+Serial Old 收集器组合回收垃圾。
4. **Serial Old 收集器**： **老年代收集器**，使用标记整理算法
 - 单线程收集器，串行，其它工作线程暂停
5. **Parallel Old 收集器**： **老年代收集器**，使用标记整理算法
 - 吞吐量优先的垃圾回收器，多线程，并行，其它工作线程暂停

6. **CMS (Concurrent Mark Sweep) 收集器**：老年代收集器，使用标记清除算法
- 多线程，致力于获取最短回收停顿时间（即缩短垃圾回收的时间），优点是并发收集（用户线程可以和 GC 线程同时工作），停顿小
 - 使用 `-XX:+UseConcMarkSweepGC` 进行 ParNew+CMS+Serial Old 进行内存回收，优先使用 ParNew+CMS，当用户线程内存不足时，采用备用方案 Serial Old 收集
-

垃圾回收 - 内存日志

内存的分配，通常与采用的 **垃圾收集器组合**、以及虚拟机中的 **内存配置** 有关

查看内存日志

```
33.125s [GC [DefNew: 3324K->152K(3712K), 0.0025925 secs] 3324K->152K(11904K), 0.0031680 secs]
100.667: [Full GC [Tenured: 0K->210K(10240K), 0.0149142 secs] 4603K->210K(19456K), [Perm : 2999K->2999K(21248K)], 0.0150007 secs] [Times: user=0.01 sys=0.00, real=0.02 secs]
```

时间

- 33.125、100.667: 表示 发生 GC 的时间 (虚拟机启动以来的秒数)
- 0.0025925: 表示 GC 的总消耗时间
- [Times:user=0.01 sys=0.00,real=0.02 secs]: 表示用户态消耗的 CPU 时间, 内核态消耗的 CPU 时间, 操作的总墙种时间

[GC、[Full GC: 表示 GC 的停顿类型

- Full GC 表示发生了一次 Stop-The-World, 而非用来区别新生代、老年代的 GC

[DefNew、[Tenured、[Perm: 表示 GC 发生的区域

- [DefNew、[ParNew、[PSYoungGen 都表示新生代, 只是垃圾收集器类型不同, 显示不同的名字
- [Tenured 表示老年代
- [Perm 表示永久代

空间

- 3324K->152K(3712K): 表示 GC 前 该区域已使用的内存 -> GC 后该区域已使用的内存(该区域的总内存)
- 方括号之外的 3324K->152K(11904K) 表示 GC 前 堆已使用的总内存 -> GC 后堆已使用的内存(堆总内存)

Heap: 表示 GC 之后代内存分配情况

```
[GC [DefNew: 6651K->148K(9216K), 0.0070106 secs] 6651K->6292K(19456K),  
0.0070426 secs] [Times: user=0.00 sys=0.00, real=0.00 secs]
```

Heap 代码执行完毕后的内存空间

```
def new generation      total 9216K, used 4326K [0x029d0000, 0x033d0000,  
0x033d0000)  
  eden space 8192K,   51% used [0x029d0000, 0x02de4828, 0x031d0000) 新生代  
  from space 1024K,   14% used [0x032d0000, 0x032f5370, 0x033d0000)  
  to   space 1024K,    0% used [0x031d0000, 0x031d0000, 0x032d0000)  
  
tenured generation      total 10240K, used 6144K [0x033d0000, 0x03dd0000,  
0x03dd0000) 老年代  
  the space 10240K,   60% used [0x033d0000, 0x039d0030, 0x039d0200, 0x03dd0000)  
  
compacting perm gen     total 12288K, used 2114K [0x03dd0000, 0x049d0000,  
0x07dd0000) 永久代  
  the space 12288K,   17% used [0x03dd0000, 0x03fe0998, 0x03fe0a00, 0x049d0000)  
No shared spaces configured.
```

设置内存配置参数

```
-verbose:gc -Xms20M -Xmx20M -Xmn10M -XX:+PrintGCDetails -  
XX:SurvivorRatio=8 -XX:PretenureSizeThreshold=3145728
```

-Xms20M: 最小堆内存 20M

-Xmx20M: 最大堆内存 20M

-Xmn10M: 10M 分配给新生代, 剩下的 10M 分配给老年代

PrintGCDetails: 打印 GC 日志

SurvivorRatio=8: Eden 和其中一个 Survivor 比例是 8:1 关系

PretenureSizeThreshold: 直接存入老年代的内存阈值（必须以 K 为单位）

实战

新生对象优先在 Eden 区分配

新生对象优先分配在新生代的 Eden 区，如果 Eden 区没有足够的空间，触发一次 Minor GC

1. bytes1/bytes2/bytes3 都分配在 Eden 区（8192K）
2. 在分配 bytes4 时发现 Eden 区内存不够，触发一次 Minor GC
3. Survivor 只有 1 M，无法存放存活的 bytes1/bytes2/bytes3，因此根据分配担保原则它们会被存放在老年代
4. bytes4 存放在 Eden 区

```
// 虚拟机配置参数: -verbose:gc -Xms20M -Xmx20M -Xmn10M -
XX:+PrintGCDetails -XX:SurvivorRatio=8
public void testAllocation() {
    byte[] bytes1, bytes2, bytes3, bytes4;
    bytes1 = new byte[2*1M];
    bytes2 = new byte[2*1M];
    bytes3 = new byte[2*1M];
    bytes4 = new byte[4*1M];           // 触发一次 Minor GC
}
```


运行结果:

```
[GC [DefNew: 6651K->148K(9216K), 0.0070106 secs] 6651K->6292K(19456K),
0.0070426 secs] [Times: user=0.00 sys=0.00, real=0.00 secs]
Heap
  def new generation      total 9216K, used 4326K [0x029d0000, 0x033d0000,
0x033d0000)
    bytes4
    eden space 8192K,  51% used [0x029d0000, 0x02de4828, 0x031d0000)
    from space 1024K,  14% used [0x032d0000, 0x032f5370, 0x033d0000)
    to   space 1024K,   0% used [0x031d0000, 0x031d0000, 0x032d0000)
  tenured generation      total 10240K, used 6144K [0x033d0000, 0x03dd0000,
0x03dd0000)
    bytes1/bytes2/bytes3
    the space 10240K,  60% used [0x033d0000, 0x039d0030, 0x039d0200, 0x03dd0000)
  compacting perm gen     total 12288K, used 2114K [0x03dd0000, 0x049d0000,
0x07dd0000)
    the space 12288K,  17% used [0x03dd0000, 0x03fe0998, 0x03fe0a00, 0x049d0000)
  No shared spaces configured.
```

大对象直接进入老年代

大对象直接进入老年代：避免在 Eden 区和 Survivor 区发生大量的复制操作

- 大对象：需要占用大量连续内存空间的对象，比如很长的字符串、数组
- **-XX:PretenureSizeThreshold**：可以设置老年代的阈值（内存大小），大于这个值的对象，直接存入老年代

```
// 虚拟机配置参数: -verbose:gc -Xms20M -Xmx20M -Xmn10M -
XX:+PrintGCDetails -XX:SurvivorRatio=8 -XX:PretenureSizeThreshold=3145728
public void testAllocation() {
    byte[] bytes = new byte[3*1M]; // 直接分配在老年代中
}
```

运行结果：

Heap

```
def new generation      total 9216K, used 671K [0x029d0000, 0x033d0000,
0x033d0000)
  eden space 8192K,      8% used [0x029d0000, 0x02a77e98, 0x031d0000)
  from space 1024K,      0% used [0x031d0000, 0x031d0000, 0x032d0000)
  to   space 1024K,      0% used [0x032d0000, 0x032d0000, 0x033d0000)
tenured generation      total 10240K, used 4096K [0x033d0000, 0x03dd0000,
0x03dd0000)
  the space 10240K, 40% used [0x033d0000, 0x037d0010, 0x037d0200, 0x03dd0000)
compacting perm gen      total 12288K, used 2107K [0x03dd0000, 0x049d0000,
0x07dd0000)
  the space 12288K, 17% used [0x03dd0000, 0x03fdefd0, 0x03fdf000, 0x049d0000)
No shared spaces configured.
```

长期存活的对象进入老年代

虚拟机给每个对象都定义了一个对象年龄计数器，每经过一次 GC，如果依然存活，并且能被 Survivor 容纳，那么计数器值加 1；当年龄增加到一定程度（默认15岁），那么就会进入老年代

- **-XX:MaxTenuringThreshold**：可以设置老年代的阈值（存活时间）

动态对象年龄判定 - 老年代

动态对象年龄判定：如果 Survivor 空间中，所有相同年龄对象的总和，大于 Survivor 空间的一半，那么年龄大于或等于该年龄的对象就可以直接进入老年代，而无视 MaxTenuringThreshold 设置的老年代年龄阈值